

μT-Kernel 3.0 BSP2

Manual Pengguna

Edisi STM32Cube

Versi 01.00.B7 | 2026.05.01

Diterjemahkan ke Bahasa Indonesia

Daftar Isi

Bab	Judul
1	Pendahuluan
1.1	Board Mikrokontroler yang Didukung
1.2	Lingkungan Pengembangan
1.3	Struktur Perangkat Lunak
1.4	Dukungan TrustZone
2	Fitur Khusus BSP
2.1	Output Serial untuk Debugging
2.2	Penggunaan File Header Standar
2.3	Dukungan FPU
3	Device Driver
3.1	Sample Device Driver A/DC
3.2	Sample Device Driver I2C
4	Prosedur Pembuatan Program
4.1	Membuat Project dengan STM32CubeMX
4.2	Integrasi μ T-Kernel 3.0 BSP2
4.3	Program Pengguna
4.4	Build dan Eksekusi
5	Riwayat Perubahan

1. Pendahuluan

Dokumen ini menjelaskan cara penggunaan μ T-Kernel 3.0 BSP2. μ T-Kernel 3.0 BSP2 adalah BSP (Board Support Package) yang memungkinkan penggunaan real-time OS μ T-Kernel 3.0 dengan memanfaatkan lingkungan pengembangan dan firmware yang disediakan oleh produsen mikrokontroler.

Dokumen ini menjelaskan μ T-Kernel 3.0 BSP2 untuk board mikrokontroler yang menggunakan STM32 dari STMicroelectronics.

1.1. Board Mikrokontroler yang Didukung

μ T-Kernel 3.0 BSP2 mendukung board mikrokontroler STM32 berikut:

Board Mikrokontroler	Mikrokontroler	CPU Core	Keterangan
NUCLEO-L476RG	STM32L476RG	Arm Cortex-M4	STMicro Nucleo-64
NUCLEO-L4R5ZI	STM32L476RG	Arm Cortex-M4	STMicro Nucleo-64
NUCLEO-F401RE	STM32F401RE	Arm Cortex-M4	STMicro Nucleo-64
NUCLEO-F411RE	STM32L411RE	Arm Cortex-M4	STMicro Nucleo-64
NUCLEO-F446RE	STM32F446RE	Arm Cortex-M4	STMicro Nucleo-64
NUCLEO-G431KB	STM32G431KB	Arm Cortex-M4	STMicro Nucleo-64
NUCLEO-G491RE	STM32G491RE	Arm Cortex-M4	STMicro Nucleo-64
NUCLEO-F767ZI	STM32F767ZI	Arm Cortex-M7	STMicro Nucleo-144
NUCLEO-H723ZG	STM32H723ZG	Arm Cortex-M7	STMicro Nucleo-144
NUCLEO-H533RE	STM32H533RE	Arm Cortex-M33	STMicro Nucleo-64
STM32N6570-DK	STM32N657X0H3Q	Arm Cortex-M55	STMicro Discovery kit

1.2. Lingkungan Pengembangan

Lingkungan pengembangan menggunakan STM32Cube IDE dari STMicroelectronics, beserta berbagai perangkat lunak STM32Cube sebagai firmware. Versi yang telah diverifikasi:

STM32CubeMX Version: 6.17.0

STM32CubeIDE Version: 2.1.1

Untuk informasi lebih lanjut, kunjungi:

STM32CubeMX: <https://www.st.com/ja/development-tools/stm32cubemx.html>

STM32CubeIDE: <https://www.st.com/ja/development-tools/stm32cubeide.html>

1.3. Struktur Perangkat Lunak

μ T-Kernel 3.0 BSP2 terdiri dari real-time OS μ T-Kernel 3.0, program dependensi untuk board mikrokontroler target, dan sample device driver. Versi μ T-Kernel 3.0 yang digunakan: v3.00.08.

Struktur direktori:

mtk3_bsp2/	(root directory)
config/	(file definisi konfigurasi)
include/	(file include)
mtkernel/	(source code OS inti)
sysdepend/	(source code bagian dependensi mikrokontroler)
stm32_cube/	(source code untuk STM32 & STM32Cube)

CPU/	(source code dependensi CPU/ARM core)
lib/	(bagian hardware-dependent dari library)
device/	(sample device driver)
doc/	(dokumentasi)

Direktori mtkernel adalah Git submodule dari μ T-Kernel 3.0 yang dipublikasikan oleh TRON Forum. Source code di bawah direktori mtkernel hanya menggunakan kode bagian umum yang tidak bergantung pada hardware. Source code yang bergantung pada board dan firmware berada di direktori sysdepend. Direktori device berisi sample program device driver yang menggunakan STM32Cube, mendukung fungsi dasar I2C dan A/DC.

1.4. Dukungan TrustZone

Untuk mikrokontroler yang memiliki fitur TrustZone dan mengaktifkannya, perhatikan hal berikut. Versi μ T-Kernel 3.0 BSP2 ini hanya mendukung eksekusi di mode Secure. Pada dasarnya dirancang hanya untuk penggunaan dengan aplikasi secure saja. Dukungan untuk aplikasi yang terdiri dari secure dan non-secure direncanakan untuk versi mendatang.

Pada pengembangan dengan TrustZone seperti STM32N657, project terdiri dari beberapa subproject: FSBL (First Stage Boot Loader), aplikasi secure, dan aplikasi non-secure. μ T-Kernel 3.0 BSP2 hanya dapat diintegrasikan ke subproject yang berjalan dalam status Secure.

2. Fitur Khusus BSP

2.1. Output Serial untuk Debugging

Output serial untuk debugging tersedia. Fitur ini dapat diaktifkan/dinonaktifkan dengan mengubah pengaturan berikut di file konfigurasi (config/config.h) dan melakukan build:

```
/*-----*/
/* Use T-Monitor Compatible API Library & Message to terminal.
 * 1: Valid 0: Invalid
 */
#define USE_TMONITOR (1) /* T-Monitor API */
```

Output serial untuk debugging menggunakan T-Monitor compatible API `tm_printf`. `tm_printf` memiliki fungsi hampir setara dengan fungsi standar C `printf`, namun tidak mendukung floating point.

Pengaturan output serial:

Item	Nilai
Baud Rate	115200 bps
Panjang Data	8 bit
Paritas	Tidak ada
Stop Bit	1 bit
Kontrol Aliran	Tidak ada

Sinyal serial terhubung ke konektor ST-LINK micro USB pada board. Peripheral yang digunakan:

Board Mikrokontroler	Peripheral
NUCLEO-L476RG	USART2
NUCLEO-L4R5ZI	LPUART1
NUCLEO-F401RE	USART2
NUCLEO-F411RE	USART2
NUCLEO-F446RE	USART2
NUCLEO-G431KB	LPUART1
NUCLEO-G491RE	LPUART1
NUCLEO-F767ZI	USART3
NUCLEO-H723ZG	USART3
STM32N6570-DK	USART1

2.2. Penggunaan File Header Standar

File header standar C dapat digunakan. Program μ T-Kernel 3.0 sendiri juga menggunakan `<stddef.h>` dan `<stdint.h>`. Dengan mengubah pengaturan berikut di file konfigurasi (config/config.h), dapat ditentukan apakah file header standar digunakan atau tidak. Jika tidak digunakan, mungkin akan terjadi error saat program pengguna meng-include file header standar.

```
/*-----*/
/* Use Standard C include file
```

```

*/
#define USE_STDINC_STDDEF (1) /* Use <stddef.h> */
#define USE_STDINC_STDINT (1) /* Use <stdint.h> */

```

2.3. Dukungan FPU

BSP ini mendukung FPU yang terpasang di dalam mikrokontroler. Informasi register FPU, dll. disertakan dalam informasi konteks task. Dengan mengubah pengaturan berikut di file konfigurasi (config/config.h), dapat ditentukan apakah FPU digunakan atau tidak. Namun, karena FSP mengaktifkan FPU, dukungan FPU pada BSP ini harus selalu diaktifkan.

```

/*-----*/
/* Use Co-Processor.
 * 1: Valid 0: Invalid
 */
#define USE_FPU (1) /* Use FPU */

```

Task yang menggunakan FPU harus menentukan TA_FPU sebagai atribut task. Namun, pada lingkungan pengembangan mikrokontroler ini, tidak memungkinkan untuk menentukan penggunaan/tidak penggunaan FPU untuk sebagian program. Oleh karena itu, jika menggunakan FPU, disarankan untuk menentukan TA_FPU pada atribut semua task.

Selain itu, dengan menetapkan berikut ke 1 di file konfigurasi, semua task dapat dibuat kompatibel FPU terlepas dari nilai atribut task:

```

#define ALWAYS_FPU_ATR (1) /* Always set the TA_FPU attribute on all tasks */

```

3. Device Driver

3.1. Sample Device Driver (A/DC)

3.1.1. Gambaran Umum

Device driver A/DC dapat mengontrol A/D converter yang terpasang di dalam mikrokontroler. BSP ini memiliki device driver A/DC yang mendukung device berikut:

Nama Device	Nama Device BSP	Keterangan
ADC	hadc	Analog-to-digital converters

Device driver ini memanfaatkan STM32CubeFW di dalam pemrosesannya. Ini adalah sample program yang menunjukkan cara menggunakan STM32CubeFW dengan μ T-Kernel 3.0, dan hanya mendukung fungsi dasar device.

Source code device driver A/DC berada di:

```
mtk3_bsp2/sysdepend/stm32_cube/device/hal_adc
```

Device driver ini dapat diaktifkan/dinonaktifkan dengan mengubah pengaturan berikut di file konfigurasi BSP (config/config_bsp/stm32_cube/config_bsp.h):

```
/* Device usage settings
 * 1: Use    0: Do not use
 */
#define DEVCNF_USE_HAL_ADC 1 // A/D conversion device
```

3.1.2. Cara Penggunaan Device Driver

(1) Konfigurasi STM32CubeFW

Konfigurasi A/D converter yang akan digunakan oleh device driver A/DC dilakukan di STM32CubeMX. Buka file dengan ekstensi .ioc di dalam project menggunakan STM32CubeMX. Atur terminal A/D converter yang akan digunakan di Pinout & Configuration → Pinout.

Referensi: Korespondensi input analog Arduino (A0~A5) dengan input A/D converter mikrokontroler untuk masing-masing board:

Input Analog Arduino	F401, F411	F446	L476	L4R5
A0	ADC1_0	ADC123_IN0	ADC12_IN5	ADC12_IN8
A1	ADC1_1	ADC123_IN1	ADC12_IN6	ADC123_IN1
A2	ADC1_4	ADC12_IN4	ADC12_IN9	ADC123_IN4
A3	ADC1_8	ADC12_IN8	ADC12_IN15	ADC123_IN2
A4	ADC1_11	ADC123_IN11	ADC123_IN2	ADC12_IN13
A5	ADC1_10	ADC123_IN10	ADC123_IN1	ADC12_IN14

Input Analog Arduino	H723	F767	G431, G491
A0	ADC12_INP15	ADC123_IN3	ADC12_IN1
A1	ADC123_INP10	ADC123_IN10	ADC12_IN2
A2	ADC12_INP13	ADC123_IN13	ADC2_IN17
A3	ADC12_INP5	ADC3_IN9	ADC3_IN12 or ADC1_IN15

A4	ADC123_INP12	ADC3_IN15	ADC12_IN7
A5	ADC3_INP6	ADC3_IN8	ADC12_IN6

Input Analog Arduino	H533	N657
A0	ADC1_INP0	ADC2_18
A1	ADC1_INP1	ADC1_10
A2	ADC1_INP5	ADC1_11
A3	ADC1_INP9	ADC1_13
A4	ADC1_INP11	ADC1_16
A5	ADC1_INP10	ADC1_8

Di Pinout & Configuration → Software Packs, pilih A/D converter yang akan digunakan dari Analog dan lakukan pengaturan. Pada Mode, atur input A/D converter yang akan digunakan ke Single-ended. Aktifkan interrupt di Configuration → NVIC Settings. Pengaturan lainnya mengasumsikan nilai default. Prioritas interrupt diatur dengan memilih NVIC dari System Core. Setelah pengaturan, klik [GENERATE CODE] untuk menghasilkan source code STM32CubeFW secara otomatis.

Catatan untuk pengguna TrustZone: Lakukan konfigurasi pada subproject yang memiliki μ T-Kernel 3.0 (pada versi ini hanya dapat berjalan di aplikasi secure).

Catatan untuk STM32N6570: TrustZone aktif. Diperlukan konfigurasi RIFSC untuk menggunakan A/D converter dari mode secure. Tambahkan kode berikut di area USER CODE BEGIN ADC1_Init 1 dalam fungsi MX_ADC1_Init:

```
/* USER CODE BEGIN ADC1_Init 1 */
__HAL_RCC_RIFSC_CLK_ENABLE();
RIFSC->RISC_SECCFGRx[2] |= 0x1;
/* USER CODE END ADC1_Init 1 */
```

(2) Inisialisasi Device Driver

Untuk menggunakan device driver A/D, pertama lakukan inisialisasi dengan fungsi dev_init_hal_adc. Dengan ini, device driver A/D yang dikaitkan dengan HAL akan dibuat. Fungsi ini didefinisikan sebagai berikut:

```
ER dev_init_hal_adc(
    UW unit,                // Nomor unit device (0 ~ DEV_HAL_ADC_UNITNM)
    ADC_HandleTypeDef *hadc // Struktur handler ADC
);
```

Parameter unit ditentukan secara berurutan mulai dari 0. Tidak boleh melompati angka. Parameter hadc adalah informasi yang dihasilkan secara otomatis oleh STM32CubeMX. Jika inisialisasi berhasil, device driver dengan nama device hadc* akan dibuat. Karakter * pada nama device diberikan huruf alfabet mulai dari 'a'. Nama device untuk unit 0 adalah hadca, unit 1 adalah hadcb, dan seterusnya.

(3) Operasi Device Driver

Device driver dapat dioperasikan menggunakan Device Management API dari μ T-Kernel 3.0. Pertama, buka device dengan menentukan nama device target menggunakan API buka tk_opn_dev. Setelah dibuka, data dapat diperoleh menggunakan Synchronous Read API tk_srea_dev. Tentukan channel A/D pada parameter posisi awal data. Device driver ini hanya dapat memperoleh 1 data dari 1 channel dalam satu akses.

Contoh program penggunaan device driver A/D:

```
LOCAL void task_1(INT stacd, void *exinf)
{
    UW adc_val1, adc_val2;
    ID dd;    // device descriptor
    ER err;   // kode error
```



```

dd = tk_opn_dev((UB*)"hadca", TD_UPDATE);    // Buka device
while(1) {
    err = tk_srea_dev(dd, 9, &adc_val1, 1, NULL);    // Baca channel 9
    err = tk_srea_dev(dd, 0, &adc_val2, 1, NULL);    // Baca channel 0
    tm_printf((UB*)"A/DC A0=%06d A/DC A1=%06d\n", adc_val1, adc_val2);
    tk_dly_tsk(1000);    // Delay 1000ms
}
}

```

3.2. Sample Device Driver (I2C)

3.2.1. Gambaran Umum

Device driver I2C dapat mengontrol device komunikasi I2C yang terpasang di dalam mikrokontroler.

Nama Device	Nama Device BSP	Keterangan
I2C	hiic	Inter-integrated circuit (I2C) interface

Source code device driver I2C berada di:

```
mtk3_bsp2/sysdepend/stm32_cube/device/hal_i2c
```

Aktifkan dengan mengubah pengaturan berikut di file konfigurasi BSP:

```

/* Device usage settings
 * 1: Use 0: Do not use
 */
#define DEVCNF_USE_HAL_IIC 1 // I2C communication device

```

3.2.2. Cara Penggunaan Device Driver

(1) Konfigurasi STM32CubeFW

Korespondensi sinyal I2C Arduino compatible interface dengan terminal I2C mikrokontroler:

Sinyal I2C Board	H533	N657	Board Lainnya
Arduino I2C SCL	PB6/I2C1_SCL	PH9/I2C1_SCL	PB8/I2C1_SCL
Arduino I2C SDA	PB7/I2C1_SDA	PC1/I2C1_SDA	PB9/I2C1_SDA

Di Pinout & Configuration → Software Packs, pilih I2C dari Connectivity. Atur mode I2C ke I2C pada Mode. Aktifkan interrupt di NVIC Settings. Klik [GENERATE CODE] untuk menghasilkan source code.

(2) Inisialisasi Device Driver

```

ER dev_init_hal_i2c(
    UW unit,    // Nomor unit device
    I2C_HandleTypeDef *hi2c    // Struktur handle I2C
);

```

Jika inisialisasi berhasil, device driver dengan nama device hiic* akan dibuat. Unit 0 = hiica, unit 1 = hiicb, dst.

(3) Operasi Device Driver

Device driver ini hanya mendukung mode controller (master) I2C. Setelah dibuka dengan tk_opn_dev, gunakan tk_srea_dev untuk menerima data dan tk_swri_dev untuk mengirim data. Tentukan alamat target (slave) pada parameter posisi awal data.

(4) Akses Register Device

Tersedia fungsi berikut untuk mengakses register device target yang terhubung via I2C:

```

/* Fungsi Register Read */
ER i2c_read_reg(

```

```

    ID dd,    // device descriptor
    UW sadr,  // alamat target
    UW radr,  // alamat register (hanya 8 bit bawah yang valid)
    UB *data  // data yang dibaca (1 byte)
);

/* Fungsi Register Write */
ER i2c_write_reg(
    ID dd,    // device descriptor
    UW sadr,  // alamat target
    UW radr,  // alamat register (hanya 8 bit bawah yang valid)
    UB data   // data yang ditulis (1 byte)
);

```

4. Prosedur Pembuatan Program

Menjelaskan prosedur dari pembuatan project program menggunakan STM32CubeMX dan STM32CubeIDE, integrasi μ T-Kernel 3.0 BSP2, build, hingga eksekusi.

4.1. Membuat Project dengan STM32CubeMX

Jalankan STM32CubeMX dan buat project board mikrokontroler target dengan langkah-langkah berikut:

- (1) Pilih menu [File] → [New Project].
 - (2) Pilih board mikrokontroler target, lalu klik [Start Project].
 - (3) Untuk mikrokontroler dengan fitur TrustZone, pilih [without TrustZone activated] untuk menonaktifkan fitur ini.

 - (3) Jika TrustZone tidak dapat dinonaktifkan, pilih [Secure domain only].
 - (4) Pilih tab [Project Manager], atur [Toolchain/IDE] di [Project] ke [STM32CubeIDE].
 - (5) Klik [GENERATE CODE] untuk menghasilkan project.
- STM32N657 tidak dapat menonaktifkan fitur TrustZone.

Pengaturan pin dan konfigurasi dasar mikrokontroler akan dilakukan. Lakukan pengaturan lainnya sesuai kebutuhan aplikasi yang dikembangkan. STM32N657 juga memerlukan pengaturan bootloader.

Catatan untuk pengguna TrustZone: Saat membuat project, atur Project Structure sebagai berikut: Secure domain only, dan centang FSBL dan Appli. Dua submodule FSBL dan aplikasi secure akan dihasilkan. Integrasikan μ T-Kernel 3.0 BSP2 ke subproject aplikasi secure.

4.2. Integrasi μ T-Kernel 3.0 BSP2

4.2.1. Import Project

Jalankan STM32CubeIDE dan import project yang dibuat dengan STM32CubeMX:

- (1) Pilih menu [File] → [Import].
- (2) Pilih [Existing Projects into Workspace] dari [General], pilih project yang dibuat dengan STM32CubeMX, lalu klik [Finish].
- (3) Project akan di-import dan tampil di [Project Explorer].

4.2.2. Integrasi Source Code

Integrasikan source code μ T-Kernel 3.0 BSP2 ke project yang dibuat. Untuk STM32N657, integrasikan ke subproject aplikasi. Dapatkan μ T-Kernel 3.0 BSP2 dari repository GitHub berikut:

https://github.com/tron-forum/mtk3_bsp2.git

Jika menggunakan perintah git, jadikan direktori project target sebagai current directory dan jalankan perintah berikut:

```
git clone --recursive https://github.com/tron-forum/mtk3_bsp2.git
```

μ T-Kernel 3.0 BSP2 menyertakan repository μ T-Kernel 3.0 sebagai Git submodule, sehingga --recursive wajib digunakan untuk mendapatkan semua source code. Direktori mtk3_bsp2 akan dimasukkan ke dalam direktori project.

Direktori μ T-Kernel 3.0 BSP2 yang dimasukkan mungkin dikecualikan dari target build STM32CubeIDE, jadi ubah propertinya. Jika Exclude resource from build dicentang, hapus centangnya.

4.2.3. Penambahan Pengaturan Build

Tambahkan pengaturan untuk membangun source code μ T-Kernel 3.0 BSP2 ke properti project. Atur di [C/C++ Build] → [Settings] → [Tool Settings]:

(1) [MCU GCC Compiler] → [Preprocessor]

Atur nama target board mikrokontroler di [Define symbols]:

Board Mikrokontroler	Nama Target
NUCLEO-L476RG	_STM32CUBE_NUCLEO_L476_
NUCLEO-L4R5ZI	_STM32CUBE_NUCLEO_L4R5_
NUCLEO-F401RE	_STM32CUBE_NUCLEO_F401_
NUCLEO-F411RE	_STM32CUBE_NUCLEO_F411_
NUCLEO-F446RE	_STM32CUBE_NUCLEO_F446_
NUCLEO-G431KB	_STM32CUBE_NUCLEO_G431_
NUCLEO-G491RE	_STM32CUBE_NUCLEO_G491_
NUCLEO-F767ZI	_STM32CUBE_NUCLEO_F767_
NUCLEO-H723ZG	_STM32CUBE_NUCLEO_H723_
NUCLEO-H533RE	_STM32CUBE_NUCLEO_H533_
STM32N6570-DK	_STM32CUBE_DISCOVERY_N657_

(2) [MCU GCC Compiler] → [Include Paths]

Atur include path berikut (asumsi direktori mtk3_bsp2 berada langsung di bawah direktori project):

```
"${workspace_loc}/${ProjName}/mtk3_bsp2"
"${workspace_loc}/${ProjName}/mtk3_bsp2/config}"
"${workspace_loc}/${ProjName}/mtk3_bsp2/include}"
"${workspace_loc}/${ProjName}/mtk3_bsp2/mtkernel/kernel/knlinc}"
```

(3) [MCU GCC Assembler] → [Preprocessor]: Lakukan pengaturan yang sama dengan (1).

(4) [MCU GCC Assembler] → [Include Paths]: Lakukan pengaturan yang sama dengan (2).

4.2.4. Pemanggilan Proses Startup OS

Project yang dihasilkan menjalankan fungsi main setelah proses inisialisasi hardware. Tambahkan kode untuk menjalankan fungsi startup μ T-Kernel 3.0, yaitu knl_start_mtkernel, dari fungsi main. Fungsi main dideskripsikan di file berikut:

<direktori project>/Core/Src/main.c

Tulis kode berikut di antara /* USER CODE BEGIN 2 */ dan /* USER CODE END 2 */ di fungsi main:

```
int main(void)
{
    /* USER CODE BEGIN 1 */
    /* USER CODE END 1 */

    /* MCU Configuration -----*/
    /** sebagian dihilangkan **/

    /* USER CODE BEGIN 2 */

    void knl_start_mtkernel(void); << tambahkan
    knl_start_mtkernel();         << tambahkan
```

```

/* USER CODE END 2 */
/** selanjutnya dihilangkan **/
}

```

4.3. Program Pengguna

4.3.1. Membuat Program Pengguna

Buat program pengguna yang akan dijalankan di μ T-Kernel 3.0. Direktori untuk membuat program pengguna bebas, baik nama maupun lokasinya. Misalnya, buat direktori bernama 'application' di level tertinggi pohon direktori project.

4.3.2. Fungsi usermain

μ T-Kernel 3.0 saat startup akan menjalankan fungsi usermain di program pengguna. Oleh karena itu, definisikan fungsi usermain dengan format berikut:

```
INT usermain(void);
```

Fungsi usermain dipanggil dari task pertama yang dibuat dan dijalankan oleh μ T-Kernel 3.0 (initial task). Jika fungsi usermain selesai, μ T-Kernel 3.0 akan shutdown. Oleh karena itu, biasanya fungsi usermain tidak diselesaikan, melainkan dibuat dalam keadaan menunggu menggunakan API penghentian task μ T-Kernel 3.0 seperti `tk_slp_tsk`. Initial task memiliki prioritas tinggi, sehingga task lain tidak akan dieksekusi selama fungsi usermain berjalan.

Jika fungsi usermain tidak didefinisikan di program pengguna, fungsi usermain default yang dideskripsikan di file berikut akan dijalankan. Fungsi usermain ini selesai tanpa melakukan apapun, sehingga μ T-Kernel 3.0 segera shutdown.

```
mtk3_bsp2/mtkernel/kernel/usermain/usermain.c
```

4.3.3. Contoh Program

Berikut contoh program pengguna. Program ini menjalankan dua task. Kedua task menampilkan string ke output serial debug pada interval tertentu.

```

#include <tk/tkernel.h>
#include <tm/tmonitor.h>

LOCAL void task_1(INT stacd, void *exinf); // fungsi eksekusi task
LOCAL ID   tskid_1;                       // nomor Task ID
LOCAL T_CTSK ctsk_1 = {                  // informasi pembuatan task
    .itskpri = 10,
    .stksz   = 1024,
    .task    = task_1,
    .tskatr   = TA_HLNG | TA_RNG3,
};

LOCAL void task_2(INT stacd, void *exinf);
LOCAL ID   tskid_2;
LOCAL T_CTSK ctsk_2 = {
    .itskpri = 10,
    .stksz   = 1024,
    .task    = task_2,
    .tskatr   = TA_HLNG | TA_RNG3,
};

LOCAL void task_1(INT stacd, void *exinf)
{
    while(1) {
        tm_printf((UB*)"task 1\n");
        tk_dly_tsk(500);
    }
}

```

```

}

LOCAL void task_2(INT stacd, void *exinf)
{
    while(1) {
        tm_printf((UB*)"task 2\n");
        tk_dly_tsk(700);
    }
}

/* fungsi usermain */
EXPORT INT usermain(void)
{
    tm_putstrstring((UB*)"Start User-main program.\n");

    /* Buat & Jalankan Task */
    tskid_1 = tk_cre_tsk(&ctsk_1);
    tk_sta_tsk(tskid_1, 0);

    tskid_2 = tk_cre_tsk(&ctsk_2);
    tk_sta_tsk(tskid_2, 0);

    tk_slp_tsk(TMO_FEVR);

    return 0;
}

```

4.4. Build dan Eksekusi

4.4.1. Build Program

Pilih project, lalu build dengan menu [Projects] → [Build project] atau menu klik kanan. Jika selesai tanpa error, file program eksekusi (file ELF) akan dihasilkan.

4.4.2. Pengaturan Debug

Lakukan pengaturan debug sesuai debugger yang digunakan. Pada board STM32 Nucleo, debugger sudah terpasang di atas board.

4.4.3. Eksekusi Program

Hubungkan board mikrokontroler ke PC, pilih menu [Run] → [Debug configurations], konfirmasi pengaturan, lalu klik tombol [Debug] untuk mentransfer program ke board dan memulai eksekusi debug. Program akan ditulis ke flash memory board mikrokontroler, sehingga jika board dinyalakan tanpa debugger, program akan langsung berjalan.

5. Riwayat Perubahan

Versi	Tanggal	Isi Perubahan
1.00.B7	2026.05.01	Penambahan informasi NUCLEO-H533. Pembaruan versi IDE dan informasi terkait.
1.00.B6	2025.05.29	Pembaruan versi OS, IDE, dll.
1.00.B5	2025.03.28	Penambahan informasi STM32N6570-DK dan TrustZone.
1.00.B4	2025.03.13	Penambahan STM32N6570-DK ke board yang didukung. Penambahan informasi terkait.
1.00.B3	2024.05.24	Koreksi kesalahan tulis.
1.00.B2	2024.04.19	Koreksi kesalahan tulis.
1.00.B1	2024.03.21	Penambahan NUCLEO-L4R5ZI ke board yang didukung. Penambahan penjelasan Bab 2. Per
1.00.B0	2023.12.15	Pembuatan baru.